

AssureX Claim Engine

Project Report

Group	NN_DevStorm
Batch	2609E2'
Faculty	Sir Minhaj
Repository	https://github.com/Saba1512006/assurex-claim-engine
Live application	https://assurex.pythonanywhere.com
Date	29 September 2026

Student ID	Name	Role
Student1525913	Saba Noor	Team lead · integration, frontend direction, QA, documentation
Student1524867	Muhammad Sami	Backend and ML pipeline developer · deployment
Student1509222	Muhammad Ghanyan	Frontend design (Figma, colour palette, mobile checks)
Student1505677	Muhammad Sami ur Rehman	Testing (sample claims, Chrome and mobile)

Abstract

AssureX Claim Engine evaluates warranty claims for consumer electronics, home appliances and industrial tools. A customer or service center registers a product from its receipt (read by OCR), files a claim with evidence, and the application pre-processes the claim into model features. A calibrated Python classification model (HistGradientBoosting, 89.3% test accuracy on 225 unseen claims) scores the structured record; the same claim is drawn as a Claim Summary Card and classified by a Google Teachable Machine model (93.8% test accuracy). The two predictions and their confidences are compared (Strong Match, Acceptable Match, Weak Match, Model Disagreement, Uncertain Result), the category's warranty policy is executed from JSON configuration, contradictions, missing documents and duplicates are detected, and a decision table produces Likely Valid, Likely Invalid or Manual Review Required. Anything uncertain goes to a human reviewer, and every step is recorded in an audit trail. The application has four roles (customer, service-center staff, claim reviewer, administrator), 295 automated tests and 34 browser tests.

The project report required by SRS §1.10 item 1 (all 35 sections, with the data flow, use case, activity, sequence and decision-flow diagrams), followed by the dataset, the warranty policy files, the security design, the demonstration video map, the AI usage declaration and the development log.

Contents

1. Project report
2. Dataset
3. Warranty policy files
4. Security and access-control design
5. Demonstration video
6. AI tool usage declaration
7. Development log

1. Project report

1. Problem definition

Manufacturers and service centers receive warranty claims that must be checked against purchase proof, product age, warranty terms, exclusions, repair history and supporting evidence. Done by hand this is slow, inconsistent between staff, and easy to defeat with reused receipts, altered dates or mismatched serial numbers. AssureX automates the checking and produces a recommendation — *Likely Valid*, *Likely Invalid* or *Manual Review Required* — that is explained, audited and sent to a person whenever the evidence is uncertain.

2. Background and business necessity

Warranty cost is a direct expense for manufacturers and a satisfaction driver for customers. Typical failure modes of manual review are: approving claims filed after the warranty and its grace period, missing exclusions (drops, liquid damage), accepting the same invoice twice, and inconsistent decisions for identical cases. An automated first pass that applies the policy the same way every time, flags contradictions and duplicates, and routes only the unclear cases to reviewers shortens turnaround and makes decisions defensible.

3. Proposed solution

A Python/Flask web application in which:

1. customers (or service-center staff for walk-in customers) register products and warranties and file claims with receipts, warranty cards, photos, videos and diagnostic reports;
2. documents are validated, fingerprinted (SHA-256) and read by OCR; the user confirms extracted values;
3. the claim is converted into a structured feature record and scored by a **Python classification model**;
4. the same record is drawn as a **Claim Summary Card** image and scored independently by a **Google Teachable Machine** model;
5. a **rule engine** checks the category's warranty policy; detectors look for **contradictions** and **duplicates**;
6. a configurable **decision table** combines rules, both predictions and their **consistency status** into the final recommendation; uncertain claims enter a **manual-review queue**;
7. reviewers decide (with overrides that require a reason), customers track progress through eight stages, and administrators see dashboards, analytics, exports and a full audit trail.

4. Purpose of the document

This report describes the requirements, design, data, models, testing and limitations of AssureX so that evaluators, maintainers and stakeholders share one understanding of what was built and how to verify it.

5. Scope

In scope: registration and authentication for four roles, product/warranty records, receipt upload and OCR, claim intake, evidence management, repair history, validation and pre-processing, the common dataset, both models, comparison, warranty rules, contradiction/duplicate/missing-document detection, claim summary and explanation, manual review, tracking, notifications and alerts,

dashboards, search, analytics, PDF reports, CSV export, audit trail, model versioning, error handling and monitoring. Out of scope (SRS §1.4): live manufacturer databases, payment systems, enterprise warranty platforms.

6. Assumptions

- Claims arrive through the web interface; a claim concerns one registered product with one warranty.
- Service-center staff belong to one service center; products may name a preferred authorised center.
- The warranty policy for a category is the JSON file in [policies/](#); changing it changes future evaluations only.
- Diagnostic confidence (0–1) comes from a technician; if nobody assessed the fault it is 0.5 (neutral).
- The training data is synthetic, generated from the policy with 4% label noise.

7. Constraints

- Quality depends on the completeness and honesty of the submitted evidence (SRS §1.5).
- The dataset size (1,500 claims) limits how much rare scenarios can be learned (e.g. late reporting).
- The two models are trained independently and do not produce identical confidences; the comparison thresholds absorb normal differences.
- Teachable Machine training happens in the browser and must be repeated by hand when cards change.
- Personal data (names, contact details, receipts) must be protected: role-scoped access, no public file URLs, audit logging, and demo data only in the repository.

8. Functional requirements (SRS §1.6) — where each is implemented

#	Requirement	Implementation
i	Registration & authentication, role-based access	src/api/auth.py , src/security/ , config/rbac.json — customer self-sign-up, other roles invited
ii	Profile, unique User ID	/profile , User.user_id (USR-...)
iii	Product registration, Product ID	/products/new , Product.product_id (PRD-...)
iv	Standard & extended warranties, provider, dates, conditions, exclusions, service center; active/expired/expiring view	ProductWarranty , product page, product list filters
v	Upload receipts/invoices/warranty cards (PDF/JPG/JPEG/PNG)	src/services/documents.py (magic-byte validation)
vi	Extract purchase date, invoice, product, model, serial, retailer, amount, warranty length	src/ocr/document_processor.py
vii	Show extracted values, let users correct them	Wizard step 3, product form auto-fill, claim page <i>Extracted details</i> (audited)
viii	Warranty dates, remaining period, active/expired/expiring/extended	ProductWarranty.status , life-used bars
ix	Expiry alerts with admin-configured window	src/services/alert_service.py , Admin > Policies > alerts

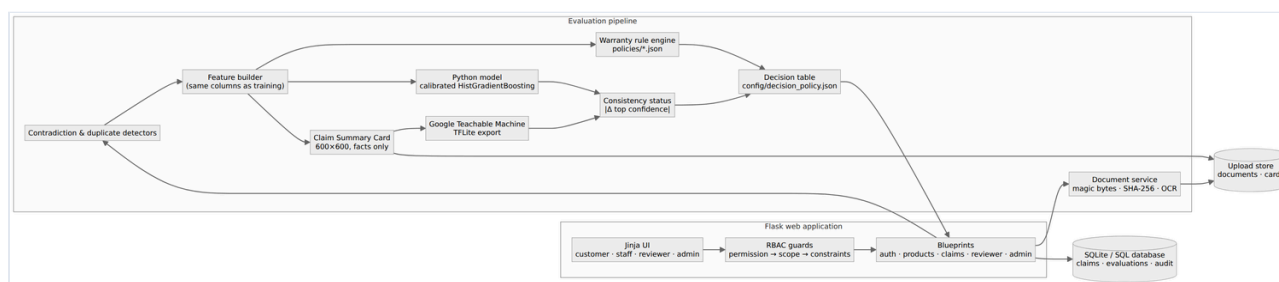
#	Requirement	Implementation
x	Claim registration by users or staff, Claim ID, links	/claims/new , Claim.claim_id (CLM-...)
xi	Claim information (age, purchase, fault date, description, cause, service history, replacement, submission date)	Wizard step 2, Claim model
xii	Photos, damage images, videos, serial photos, diagnostic reports	Eight evidence types incl. MP4 video
xiii	Repair history (dates, center, parts, outcome, cost, authorised or not)	RepairHistory , product page (staff)
xiv	Documents organised per product and claim; view/download/replace/remove by rights	Product and claim evidence panels, document.* permissions
xv	Validation: mandatory fields, dates, numbers, file types, sizes, duplicate IDs	src/rules/validator.py , documents.validate , unique DB constraints
xvi	Pre-processing: missing values, dates, encoding, normalisation, derived fields	src/core/features.py + saved sklearn pipeline
xvii	Common dataset in CSV and card form	data/ , dataset_generator/
xviii	Train & compare algorithms, select best	notebooks/train_python_v2.py
xix	Python confidence for all three classes	Claim page, stored per evaluation
xx	Claim Summary Card without prediction/decision	src/core/card_v2.py
xxi	Teachable Machine classification	src/core/gtm_classifier_v2.py
xxii-xxiv	Class match, Δ confidence , five consistency statuses with configurable thresholds	src/core/consistency.py , config/decision_policy.json
xxv-xxvi	Warranty rule validation, configurable per category	src/rules/policy_engine.py , policies/*.json , Admin > Policies
xxvii	Serial number vs receipt, warranty card, product image, repair records	src/rules/contradiction_detector.py
xxviii	Contradictions (dates, models, serials)	same
xxix	Missing mandatory documents, telling the user which	Policy mandatory_documents / supporting_documents , checklist, notifications
xxx-xxxi	Duplicate claims and documents (IDs, invoices, serials, descriptions, claimant, hashes, history)	src/rules/duplicate_detector.py
xxxii	Claim summary	src/services/explain.py → summarise (template-based, no external AI)
xxxiii	Claim preparation assistance	Live readiness checklist in the wizard
xxxiv	Final decision from all factors	src/core/pipeline.py , decision table
xxxv	Explanation: supporting/opposing factors, rules passed/failed, contradictions, evidence needed	explain.py → explain , claim page
xxxvi-xxxvii	Manual-review queue, approve/reject/request info, comments, override with reason, original AI result kept	src/api/reviewer.py , ReviewerAction
xxxviii	Eight tracked stages	Claim.transition_status , stage bar, tracking page
xxxix	Notifications (expiry, submission, missing docs, info requests, status, review, approval, rejection)	src/services/notifications.py , bell menu, dashboard
xl	User dashboard	/claims/
xli	Admin dashboard (totals, classes, pending, duplicates, disagreements, average confidences, trend)	/admin/dashboard

#	Requirement	Implementation
xlii	Search & filter by claim, product, category, serial, warranty status, claim status, risk, confidence, reviewer, dates	/claims/search
xliii	Analytics (outcomes, faults, rejection reasons, categories, expirations, repairs, model performance, manual-review frequency)	/admin/analytics
xliv	Downloadable claim report	<a href="/claims/<id>/report.pdf">/claims/<id>/report.pdf (src/services/report_generator.py)
xlvi	CSV/Excel export	<a href="/admin/export/<kind>">/admin/export/<kind> (CSV with BOM, or ?format=xlsx workbook); formula-injection guard in both
xlvi	Secure database storage	SQLAlchemy models, SQLite/any SQL
xlvi	Audit trail	AuditLog , src/services/audit.py , Admin > Audit
xlviii	Model version tracking; updates don't alter recorded results	Hash-based versions on every ModelEvaluation row; rows never updated
xlix	Understandable errors	Friendly error pages and flash messages; model/OCR failures degrade gracefully
I	Monitoring & anomaly alerts	alert_service.anomalies() on the admin dashboard; <i>Send due alerts</i>

9. Non-functional requirements

NFR	Result
Performance ≤ 5 s	Full evaluation (rules + Python + card + image model + decision) ~150–300 ms on a laptop; stored per claim (latency_ms) and tested
Scalability (10,000 claims, several centers, concurrent users)	Indexed columns (claim_id, status, reviewer, serial, hashes); scoped SQL queries; stateless workers; any SQL database via DATABASE_URL
Usability and accessibility	Four-step wizard with a live readiness checklist and server-saved drafts, plain-language explanations, responsive layout from 390 px, keyboard shortcuts for reviewers, visible focus, reduced-motion support. axe-core finds no serious or critical WCAG 2 A/AA issue on 21 pages; Lighthouse scores 100 for accessibility, best practices and SEO and 99 for performance on the public pages
Accuracy $\geq 85\%$	Python model 89.3%, Teachable Machine 93.8% on the 225 unseen test claims
Availability $\geq 99\%$	Stateless app behind gunicorn; /healthz for uptime monitors; no runtime dependency on external APIs or CDNs

10. Application architecture



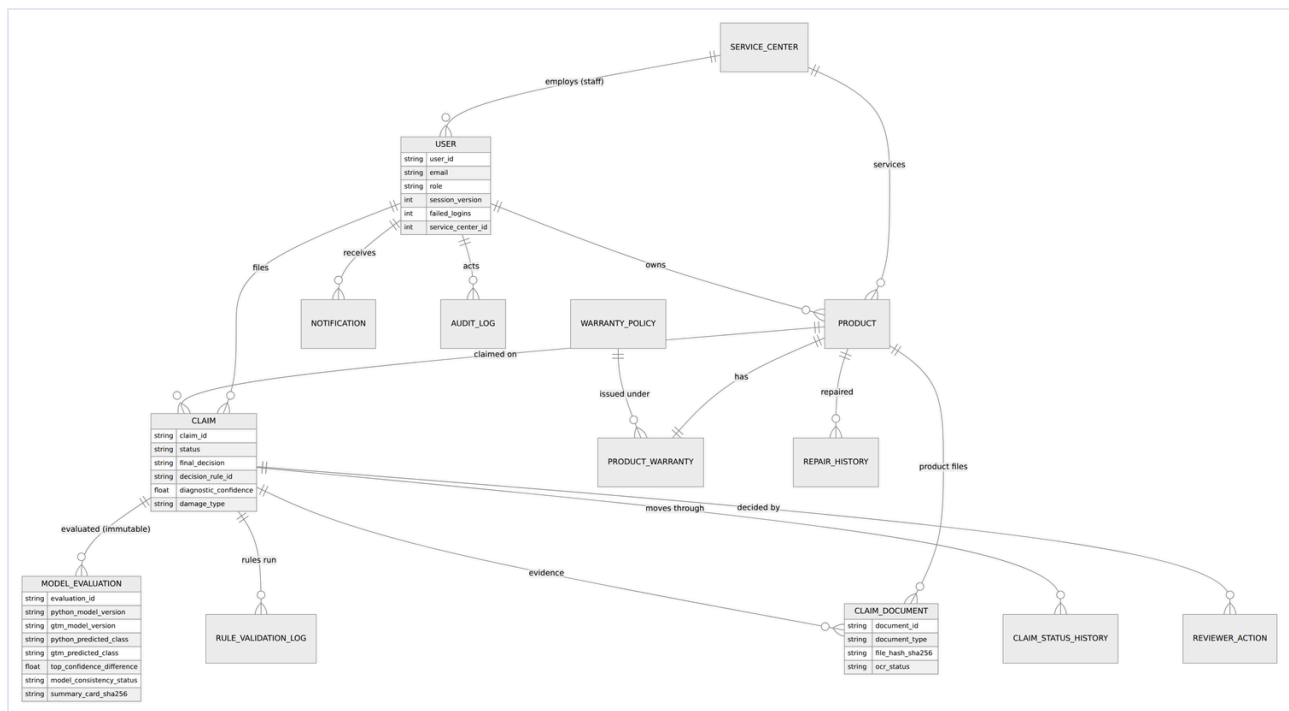
Architecture

Three layers: the **web layer** (Jinja templates on one component library, design tokens in [static/css/tokens.css](#), self-hosted IBM Plex fonts, a shell script plus small page scripts and the Alpine.js CSP build; the Content-Security-Policy allows only same-origin scripts and one nonce'd bootstrap line, and every JSON endpoint answers with one `{success, data, error}` envelope), the **application layer** (Flask blueprints guarded by the RBAC engine; services for the claim lifecycle, documents, alerts, analytics, exports and reports) and the **evaluation layer** (feature builder, rule engine, detectors, both model runtimes, card renderer, consistency and decision table). Persistence is SQLAlchemy over SQLite by default plus a file store for documents and cards.

11. Module descriptions

Module	Responsibility
src/core/vocab.py	Single vocabulary: categories, damage types, faults, classes, document types, date formats
src/core/features.py	Pre-processing and feature engineering for a stored claim
src/core/python_classifier.py	Loads the versioned sklearn pipeline; never retrains itself
src/core/card_v2.py	Deterministic 600×600 card renderer (v3 tile design) with bundled fonts
src/core/gtm_classifier_v2.py	Teachable Machine TFLite runtime, label parsing, version hashing
src/core/consistency.py , decision_table.py	Five statuses; first-match decision table with validation and safe saving
src/core/pipeline.py	Orchestration and immutable persistence of each evaluation
src/core/offline_eval.py	Same pipeline over the val/test splits for evaluation and reports
src/rules/policy_store.py	Loads/validates/saves policy JSON; rule catalogue
src/rules/policy_engine.py	Sixteen rule checks; severity from policy
src/rules/contradiction_detector.py , duplicate_detector.py	Evidence consistency and duplicate indicators
src/rules/validator.py	Form validation and preparation checklist
src/security/rbac.py , guards.py	Permission → scope → constraint engine; request hooks, decorators, scoped queries, login hardening
src/services/*	Claim lifecycle, documents, notifications, audit, alerts, analytics, exports, PDF report, summaries
src/api/*	Public, auth, products, claims, reviewer (queue and workbench), admin, access-control blueprints
src/api/api.py , errors.py	JSON endpoints (stored verdict, live bench, draft autosave, what-if, batch) and the error vocabulary
src/core/explain_models.py	Per-claim explanations: Python feature contributions (replace each input with its training median/mode) and Teachable Machine tile occlusion
src/services/verdict.py	Verdict screen data from the stored payload: agreement-meter geometry, reviewer overrides, customer visibility
src/services/whatif.py , batch.py	Threshold simulator over the labelled test claims and live claims; chunked batch evaluation of uploaded CSVs
src/services/model_card_service.py	Public model card: metrics, calibration (reliability bins, ECE), per-category accuracy, label-noise analysis

12. Database design



Entity relationships

Sixteen tables. Key points: public identifiers (USR-, PRD-, CLM-, DOC-, EVL-) are random and unique; **claims** carries status, recommendation, decision rule and flags; **model_evaluations** stores the exact feature record, both predictions with all three confidences, model versions, card hash, thresholds and the decision trace — one row per run, never updated; **rule_validation_logs** stores every rule with its severity and message plus contradictions, duplicate indicators and missing documents; **claim_status_histories** and **audit_logs** are append-only; **users** hold security state (session version, failed logins, logout, forced password change, service center).

13. Data dictionary

Dataset data dictionary (data/data_dictionary.json)

Column	Description
claim_id	Claim identifier CLM-00001..CLM-01500, assigned after shuffling (never encodes the class). Not a feature.
product_id	Product identifier PRD-xxxxx. Not a feature.
product_category	Consumer Electronics Home Appliances Industrial Tools (src/core/vocab.py). Feature (one-hot).
product_brand	Descriptive. Not a feature.
product_model	Descriptive. Not a feature.
product_serial	Descriptive. Not a feature.
retailer	Descriptive. Not a feature.
invoice_number	Descriptive. Not a feature.
fault_category	Fault from the category's covered-fault list. Used by the rule engine (UNCOVERED_FAULT), not a model feature.
fault_description	Free text written from fault + cause, class-independent. Never a feature.
damage_type	Customer-describable cause (7 values, vocab.DAMAGE_TYPES). Feature (one-hot). Not a relabelled class.

Column	Description
purchase_date	YYYY-MM-DD.
warranty_start_date	Equal to purchase date.
warranty_expiry_date	purchase_date + round(months * 30.44) days.
fault_occurrence_date	YYYY-MM-DD; before purchase_date when claim_date_conflict_flag = 1.
claim_submission_date	YYYY-MM-DD within 2026.
warranty_duration_months	Standard term of the category (+12 when extended). Feature.
is_extended_warranty	1 if an extended term was bought. Feature.
purchase_price	Log-normal price. Feature.
product_age_days	claim date - purchase date. Feature.
days_to_expiry	Coverage days - age; negative once expired. Feature.
remaining_warranty_days	max(0, days_to_expiry). Display only.
reporting_delay_days	claim date - fault date (0 if negative). Feature.
diagnostic_confidence	Technician's confidence (0-1) that the stated cause is right; 0.5 = not assessed. Feature.
previous_repairs_count	Earlier repairs on record. Feature.
unauthorized_repair_flag	1 if any earlier repair was by an unauthorised center. Feature.
serial_number_match	0 if the registered serial differs from the evidence. Feature.
duplicate_invoice_flag	1 if the invoice/receipt was already used elsewhere. Feature.
claim_date_conflict_flag	1 if the dates contradict (fault before purchase, etc.). Feature.
has_receipt	Purchase receipt present (0/1). Feature.
has_warranty_card	Warranty card present. Feature.
has_damage_photo	Damage/fault photo present. Feature.
has_serial_photo	Serial photo present. Feature.
has_repair_report	Diagnostic/repair report present. Feature.
missing_document_count	4 - (receipt + warranty card + damage photo + serial photo). Feature.
scenario	Why the ground-truth policy gave this label (e.g. expired, grace_boundary, covered_defect); '+noise' marks the 4% flipped labels. Never a feature.
claim_class	Target: Valid Claim Invalid Claim Manual Review.

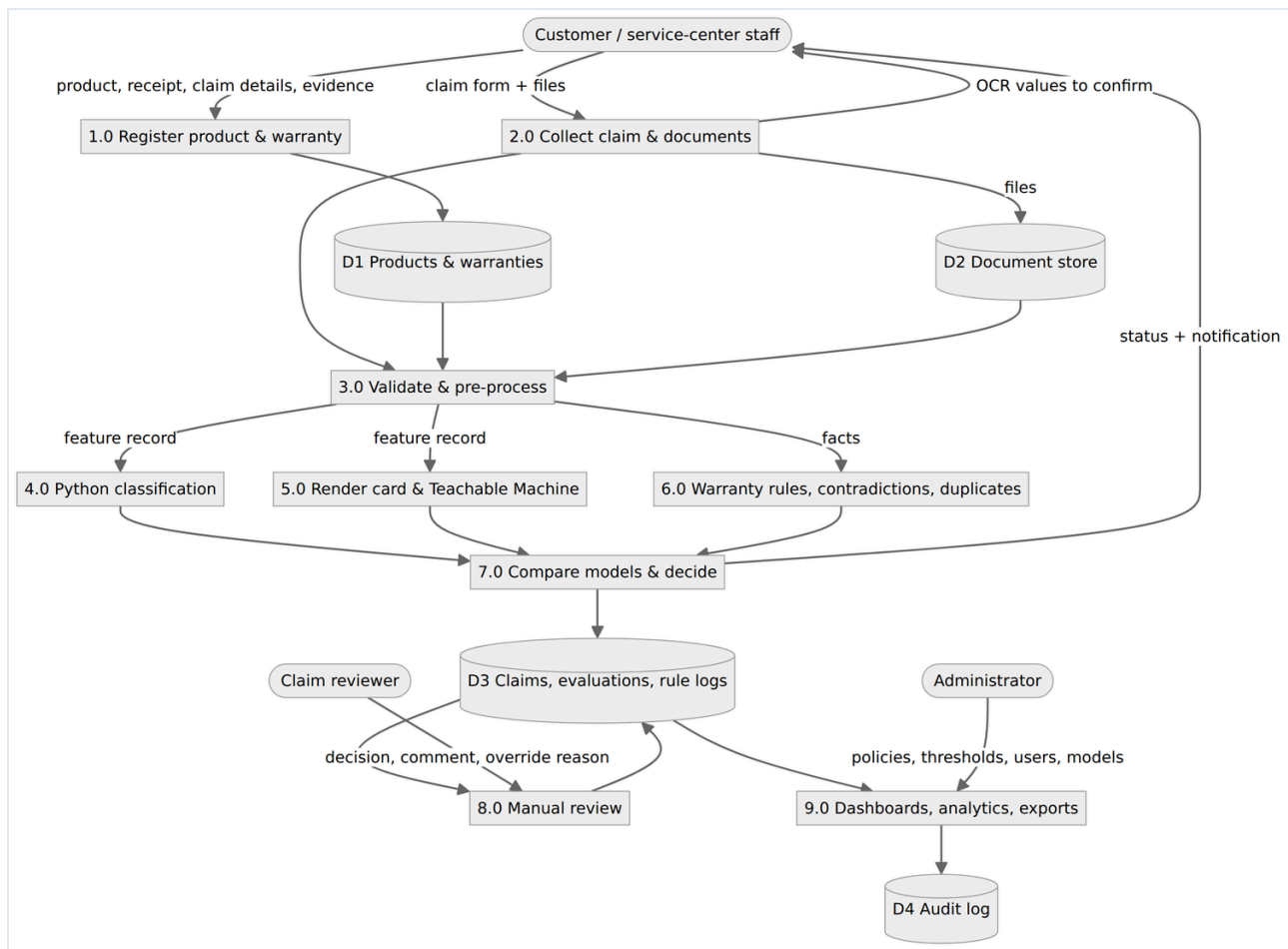
Database tables (16, [src/models/entities.py](#))

Table	Columns	Fields (PK primary key, FK foreign key)
service_centers	6	id (PK), center_id, name, city, phone, created_at
users	17	id (PK), user_id, email, password_hash, full_name, role, phone_number, address, is_active, session_version, failed_logins, locked_until, must_change_password, last_login_at, service_center_id (FK → service_centers), created_at, updated_at
products	14	id (PK), product_id, user_id (FK → users), service_center_id (FK → service_centers), product_name, category, brand, model_number, serial_number, purchase_date, purchase_price, retailer, invoice_number, created_at
warranty_policies	10	id (PK), policy_id, category, policy_name, coverage_duration_months, grace_period_days, claim_reporting_period_days, authorized_service_center_required, policy_rules_json, created_at
product_warranties	14	id (PK), warranty_id, product_id (FK → products), policy_id (FK → warranty_policies), warranty_provider, start_date, expiry_date, duration_months, is_extended, extended_months, coverage_conditions, exclusions, service_center_name, created_at

Table	Columns	Fields (PK primary key, FK foreign key)
claims	28	id (PK), claim_id, user_id (FK → users), created_by_id (FK → users), product_id (FK → products), warranty_id (FK → product_warranties), service_center_id (FK → service_centers), fault_occurrence_date, fault_description, fault_category, damage_type, diagnostic_confidence, diagnosis_source, previous_replacement_details, claim_submission_date, status, risk_level, final_decision, decision_reason, decision_rule_id, decision_policy_version, is_duplicate_flag, contradiction_flag, missing_document_flag, assigned_reviewer_id (FK → users), reviewer_notes, created_at, updated_at
claim_documents	16	id (PK), document_id, claim_id (FK → claims), product_id (FK → products), uploaded_by_id (FK → users), document_type, file_path, original_filename, mime_type, file_size_bytes, file_hash_sha256, ocr_status, ocr_extracted_text, ocr_data_json, verified_by_user, created_at
repair_histories	14	id (PK), repair_id, product_id (FK → products), claim_id (FK → claims), recorded_by_id (FK → users), repair_date, repair_center, replaced_parts, outcome, repair_cost, is_authorized_center, serial_number_seen, notes, created_at
notifications	10	id (PK), notification_id, user_id (FK → users), notification_type, title, message, related_claim_id, related_product_id, is_read, created_at
claim_status_histories	7	id (PK), claim_id (FK → claims), previous_status, new_status, changed_by_user_id (FK → users), reason_comment, created_at
model_evaluations	30	id (PK), evaluation_id, claim_id (FK → claims), features_json, python_available, python_model_version, python_predicted_class, python_conf_valid, python_conf_invalid, python_conf_manual, gtm_available, gtm_model_version, gtm_predicted_class, gtm_conf_valid, gtm_conf_invalid, gtm_conf_manual, model_errors_json, is_class_match, top_confidence_difference, model_consistency_status, consistency_thresholds_json, decision, decision_rule_id, decision_trace_json, decision_policy_version, summary_card_image_path, summary_card_sha256, latency_ms, payload_json, evaluation_timestamp
rule_validation_logs	10	id (PK), claim_id (FK → claims), evaluation_id (FK → model_evaluations), policy_name, rules_json, contradictions_json, duplicate_flags_json, missing_documents_json, overall_rule_status, execution_timestamp
reviewer_actions	10	id (PK), claim_id (FK → claims), reviewer_id (FK → users), previous_status, previous_recommendation, reviewer_decision, is_override, override_reason, comments, timestamp
audit_logs	9	id (PK), user_id (FK → users), user_role, action, entity_type, entity_id, ip_address, details_json, timestamp
system_settings	5	id (PK), setting_key, setting_value, description, updated_at
batch_runs	12	id (PK), batch_id, created_by_id (FK → users), filename, status, total, processed, rows_json, results_json, error, created_at, finished_at

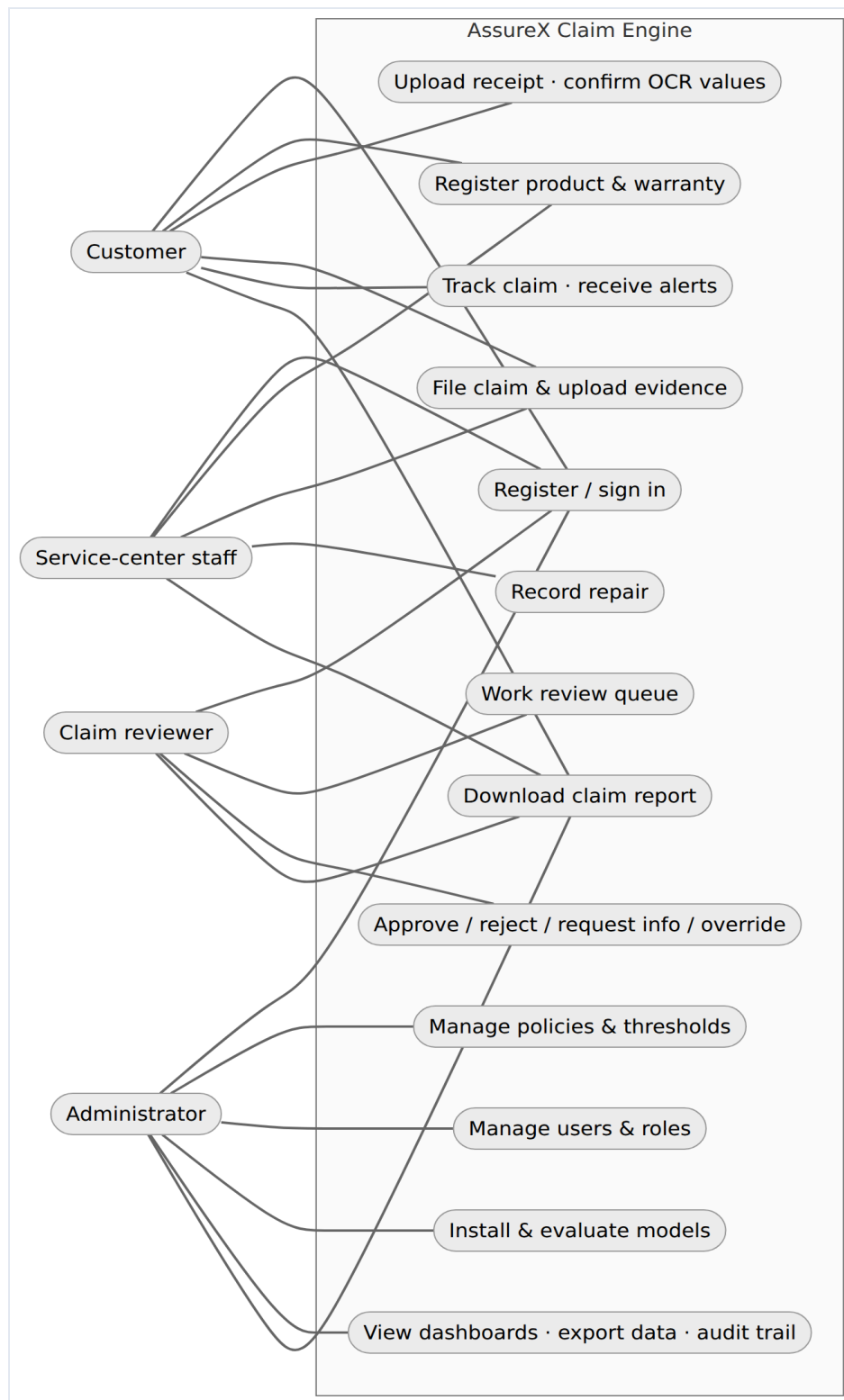
The dataset columns are documented in [data/data_dictionary.json](#) (37 columns: identifiers, descriptive fields, 20 model features, scenario and label). Database fields are documented in [src/models/entities.py](#).

14. Data flow diagram



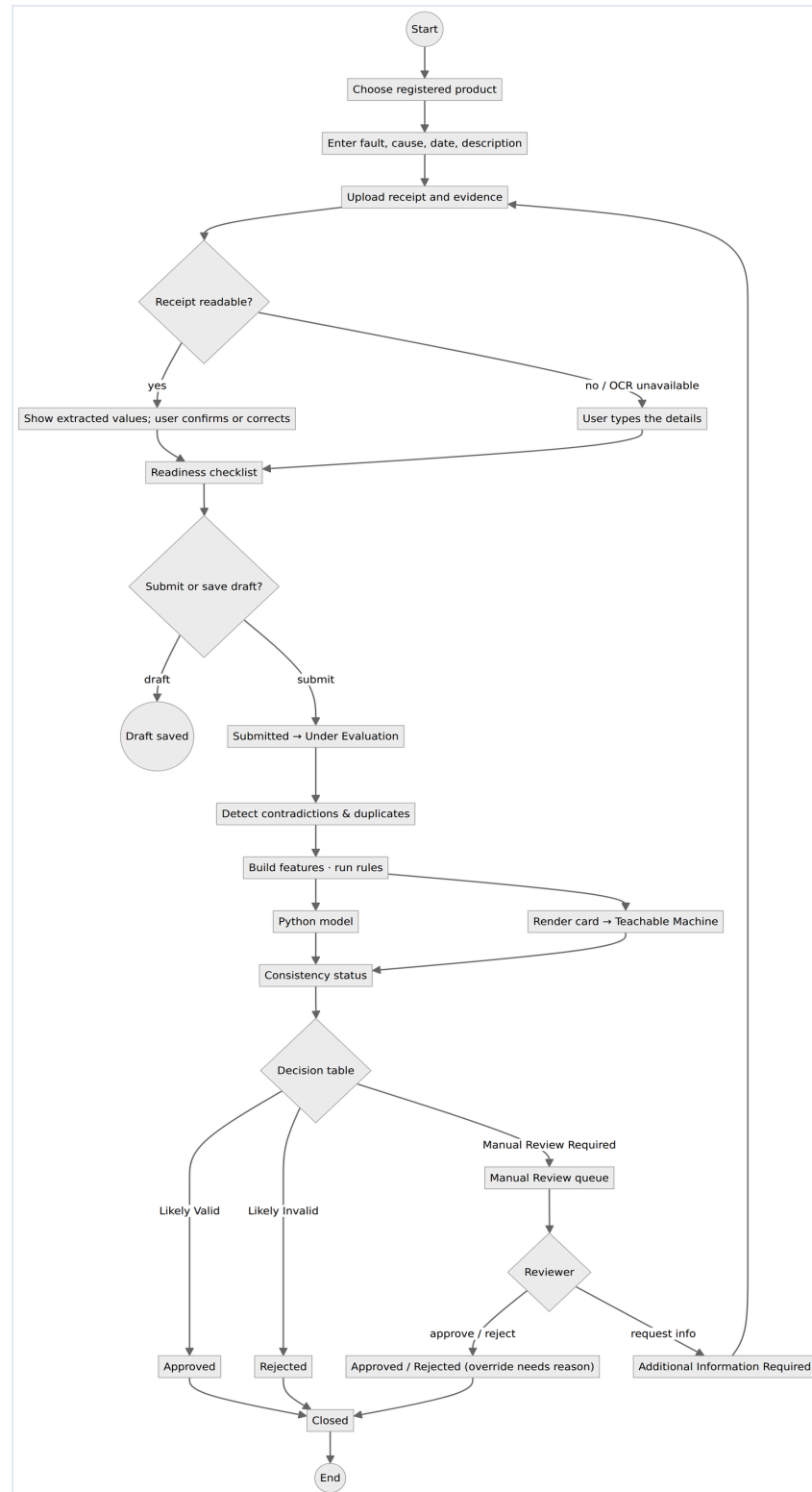
Level-1 DFD

15. Use case diagram



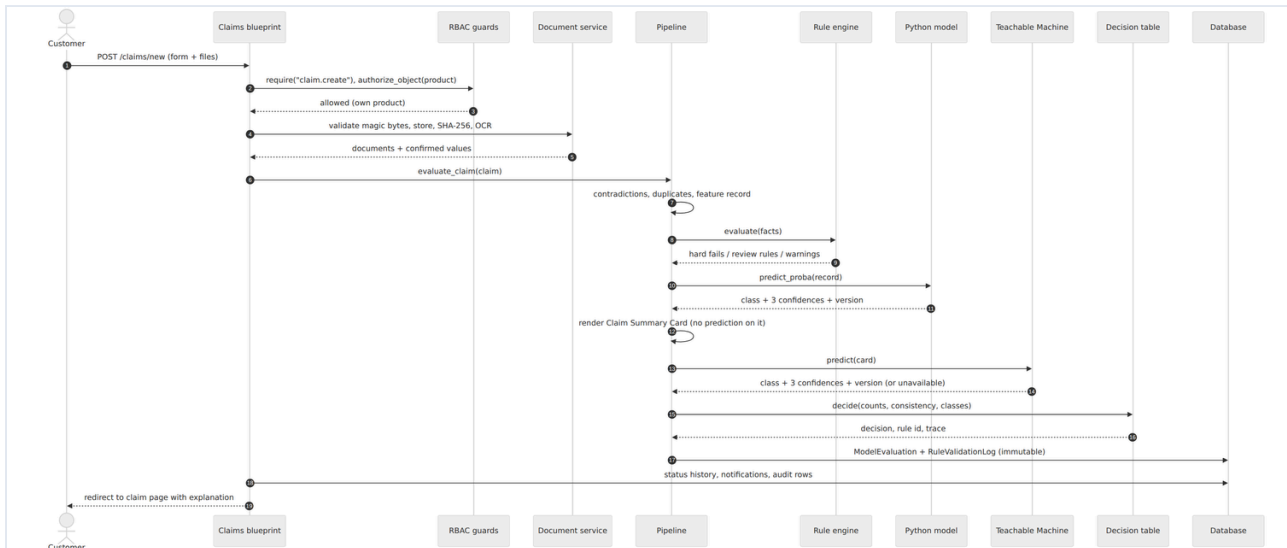
Use cases

16. Activity diagram



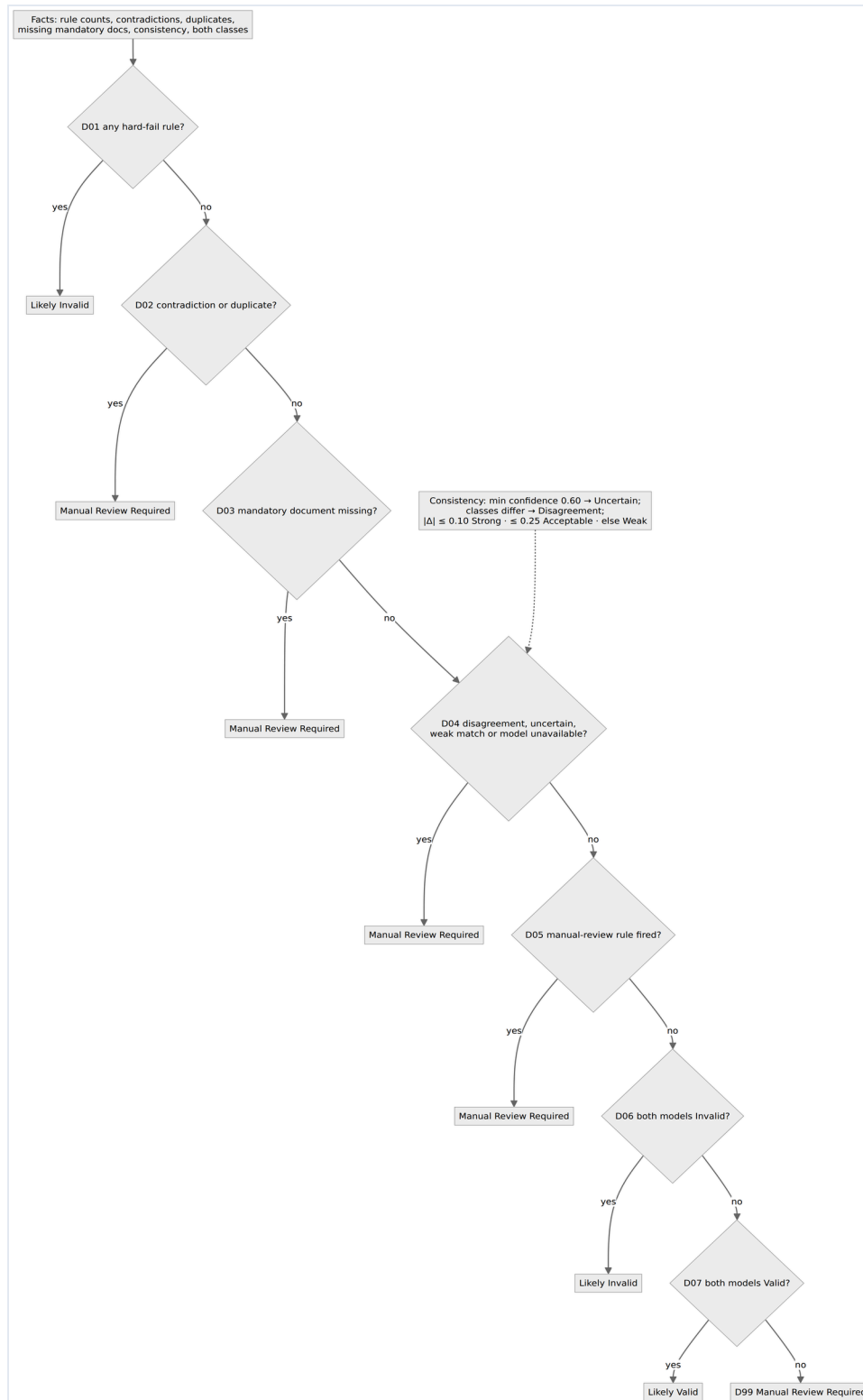
Claim activity

17. Sequence diagram



Evaluation sequence

18. Decision flow diagram



Decision flow

19. Rule-engine design

Each category has a JSON policy with coverage, standard terms, extension length, start conditions, grace period, reporting period, covered faults, excluded damage causes (plus human-readable exclusions), diagnosis threshold for confirming an exclusion, repeat-repair threshold, repair and replacement conditions, authorised-center requirement, mandatory and supporting documents, and three rule lists: `hard_fail_rules`, `manual_review_rules`, `warning_rules`. The engine has sixteen checks

(e.g. WARRANTY_EXPIRED, GRACE_PERIOD, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING, INCOMPLETE_EVIDENCE); which list a rule is in decides its severity, and a rule in no list is off. Every result — fired or passed — is stored with a message, so the explanation can show rules passed and failed. Files are validated against the vocabulary on load and on every admin save (atomic write, audited).

20. Python classification model design

Feature record (8 numeric, 10 binary, 2 categorical) → [ColumnTransformer](#) (scaler for the linear model, one-hot with fixed categories) → candidate classifier. Three candidates compared with stratified 5-fold CV and a validation split; the best on validation macro-F1 is wrapped in sigmoid calibration (5-fold) and saved as one pipeline artefact with its feature list and library version. Full evidence:

[documentation/PYTHON_MODEL_EVIDENCE.md](#).

21. Google Teachable Machine model design

Standard image model trained on 2,100 Claim Summary Cards (700 per class; two variations of each training claim), exported as TensorFlow Lite. The card (v3) shows each claim fact against its policy limit as one of nine large colour-and-glyph tiles in a fixed 3 × 3 grid (coverage, reporting time, damage cause, receipt, supporting evidence, serial, dates, invoice, repairs), because Teachable Machine classifies with frozen ImageNet features and cannot read small text. Procedure and status:

[documentation/GTM_EVIDENCE.md](#).

22. Dataset description

1,500 claims, exactly 500 per class, across three categories (Consumer Electronics 464, Home Appliances 514, Industrial Tools 522). Scenarios (from the ground-truth policy): covered defect 487, expired 223, missing documents 144, ambiguous cause 127, unauthorised repair 98, grace boundary 80, excluded damage 76, serial mismatch 60, contradiction 51, repeat repairs 35, possible duplicate 31, late reporting 23, plus 63 noise-flipped labels. This covers the SRS's normal, incomplete, contradictory, complex and borderline cases.

23. Dataset-generation method

[dataset_generator/generator_v2.py](#): attributes are sampled from realistic, overlapping distributions independently of the class (where in the warranty life the claim falls, reporting delay, damage cause by category, diagnosis, repairs, documents, serial match, duplicate invoice, date conflict). The label is then **derived** by an adjudication function that mirrors the policy files, and 4% of labels are flipped to model reviewer disagreement. Records are drawn until each class has 500, shuffled, and only then given claim IDs. A stratified 70/15/15 split follows, before any card is rendered. v1 had a [damage_type](#) column that was a renamed label and claim IDs ordered by class; both are fixed and tested.

24. Data pre-processing and feature engineering

Dates parsed with seven accepted formats; derived fields (product age, days to expiry, remaining warranty, reporting delay, missing-document count, repair count, unauthorised-repair / serial-mismatch / duplicate-invoice / date-conflict flags); missing documents → 0; unassessed diagnosis → 0.5; categorical values validated against the vocabulary and one-hot encoded with fixed categories; numeric scaling inside the pipeline for the linear candidate.

25. Python model-training procedure

`python notebooks/train_python_v2.py`: leakage audit (fails above 90%) → 5-fold CV and validation metrics for each candidate → select on validation → permutation importance → calibrate → one-time test evaluation (accuracy, macro precision/recall/F1, ROC-AUC, log-loss, ECE, confusion matrix, per-class report, 30 sample predictions) → save pipeline and model card. Asserts accuracy ≥ 85%.

26. Google Teachable Machine training procedure

See `documentation/GTM_EVIDENCE.md` §“Training procedure”. After export the model is uploaded in Admin > Models, validated, versioned, and evaluated on the 450 held-out cards.

27. Model evaluation

Model	Accuracy	Macro precision	Macro recall	Macro F1	ROC-AUC
Python (HistGradientBoosting, calibrated), test	89.3%	89.5%	89.3%	89.2%	0.948
Teachable Machine (<code>gtm-514702678b8c</code>), test	93.8%	94.0%	93.8%	93.8%	—

28. Confusion matrix (Python model, test)

actual \ predicted	Invalid	Manual Review	Valid
Invalid Claim	61	8	6
Manual Review	2	69	4
Valid Claim	3	1	71

29. Accuracy, precision, recall and F1 by class (Python model, test)

Class	Precision	Recall	F1
Invalid Claim	92.4%	81.3%	86.5%
Manual Review	88.5%	92.0%	90.2%
Valid Claim	87.7%	94.7%	91.0%

30. Model prediction and confidence comparison

`reports/generate_comparison_report.py` produces `reports/model_comparison_report.csv/.md` for the test claims with every SRS column (IDs, actual class, both predictions and confidences, card filename, match, difference, status, rule result, missing documents, contradictions, duplicates, final decision, explanation of disagreements, summary). Consistency rules: minimum top confidence 0.60 (else *Uncertain Result*); different classes → *Model Disagreement*; $|\Delta| \leq 0.10$ *Strong*, ≤ 0.25 *Acceptable*, otherwise *Weak Match*; the latter three statuses and disagreement all route to manual review. On the test claims the two models agree on 92.9%. If the Teachable Machine export is missing, the report states that its columns are unavailable rather than estimating them.

Every stored verdict also carries its explanation: the three inputs that moved the Python model most (each replaced by its typical training value) and the Claim Summary Card tiles that changed the image model's answer most when blanked out. Administrators can replay all 225 test claims under different agreement thresholds in the what-if simulator before changing them; applying a change saves a new decision-policy version with its simulated effect in the audit trail.

Design check with a simulated agreeing image model (not a reported metric): the rules + decision table map 93.3% of test claims to their labelled class, and 10 of the 15 remaining differences are the deliberately noisy labels — i.e. the combination is sound once the second model is in place.

31. Testing strategy

295 automated pytest tests across unit (rules, detectors, OCR patterns, consistency, RBAC policy), integration (full HTTP journeys through the real app, DB and file store), boundary, negative, security, database, model and demonstration cases; surprise-modification scenarios are tests too. A generated route × role matrix calls every guarded route (68 route/method pairs) as each role and signed out and compares the answer with [config/rbac.json](#). Accuracy guards re-render the test cards with the current code and fail if the image model's accuracy could drop. A separate browser suite (`python -m pytest -m e2e tests/e2e`, 34 tests) runs the main journeys on a live seeded server with the real models, checks accessibility with axe, keyboard use, reduced motion, phone width and session revocation, and fails on any console error. Catalogue: [documentation/TEST_CASES.md](#); screenshots in [docs/screenshots/](#).

32. Security considerations

Deny-by-default RBAC with record scopes and separation of duties; out-of-scope records return 404; user state reloaded per request with session versioning (role change/disable signs out everywhere); idle timeout; generic login errors, five-attempt lockout (one minute, with a countdown on the sign-in page), session rotation, rate-limited sign-in; server-side input rules (no digits in names, no letters in phone numbers, strict email format); public sign-up limited to customers; forced password change for invited users; CSRF tokens on all forms; strict CSP, clickjacking and MIME-sniffing protection; uploads validated by magic bytes, stored under random names outside the web root and served only through permission checks; CSV formula-injection guard; parameterised queries via the ORM; secrets from the environment; append-only audit including denied access. Design and v1 loophole list: [documentation/RBAC_DESIGN.md](#).

33. Privacy considerations

Only data needed to decide a claim is collected. Each role sees only its scope; documents are never publicly addressable; exports require an explicit permission and are audited. The repository contains only fictitious demo data and synthetic images. Deployments should use HTTPS (secure cookies are on by default), restrict database backups, and define a retention period for closed claims and uploaded files.

34. Limitations

- The Teachable Machine model must be retrained in the browser whenever the card design or policy limits change.
- Synthetic training data; rare scenarios (late reporting) are learned less well; real data needs retraining.
- Image OCR requires Tesseract on the server.
- Duplicate text similarity is lexical ($\geq 90\%$ sequence match), not semantic.
- SQLite and in-memory rate limits suit a single server; use PostgreSQL/Redis for multiple workers.
- The saved Python model needs scikit-learn 1.9.1 (Python 3.11+). On the free PythonAnywhere account used for the live demo (Python 3.10, 512 MB disk) the models can be unavailable, and claims then go to manual review.

35. Future enhancements

Retrain on real claims with periodic recalibration and drift monitoring; feed the reviewer-decision export (Admin > Overview) back into training; image-based damage detection on customer photos; semantic duplicate detection; email/SMS notifications; multi-tenant manufacturer support; REST API for service-center systems; database migrations with Alembic.

2. Dataset

One common dataset of synthetic warranty claims in two linked forms: structured CSV records for the Python model and Claim Summary Card images for Google Teachable Machine. Each CSV record and its card carry the same Claim ID, claim details and class label. Splits are made before any card is rendered, so a validation or test claim never appears in training in either form.

Item	File
Raw dataset (1,500 claims)	data/raw/common_warranty_claims_1500.csv
Training / validation / test CSV	data/splits/train.csv, val.csv, test.csv (1,050 / 225 / 225)
Model-ready feature matrices	data/processed/{train,val,test}_features.csv
Card images	data/summary_cards/train/{valid,invalid>manual_review}/ (700 each), val/ (225), test/ (225)
Claim ID ↔ image mapping	data/claim_id_to_card_mapping.csv
Labels	data/labels.csv
Scenario definitions	data/claim_scenarios.json
Statistics	data/dataset_statistics.json
Data dictionary	data/data_dictionary.json
Generation scripts	dataset_generator/generator_v2.py, dataset_generator/build_cards_v2.py

Dataset statistics

Class	Claims
Manual Review	500
Valid Claim	500
Invalid Claim	500

By category

Value	Count
Industrial Tools	522
Home Appliances	514
Consumer Electronics	464

Claim scenarios and labelling rules

rules are applied in this order; the first that matches decides the label. 65 claims (4.3%) carry a deliberately flipped label (scenario suffix '+noise') to simulate reviewer disagreement.

Scenario	Class	Definition	Claims
expired	Invalid Claim	Claim filed after the warranty and its grace period ended.	237
unauthorized_repair	Invalid Claim	The product was previously repaired by an unauthorised center.	102
excluded_damage	Invalid Claim	Damage cause is excluded by the category policy and the technician's diagnosis confirms it (confidence at or above the policy threshold).	81

Scenario	Class	Definition	Claims
serial_mismatch	Invalid Claim	Serial number differs from the receipt and the serial photo.	63
contradiction	Manual Review	Dates contradict each other (fault reported before purchase).	53
late_reporting	Invalid Claim	Fault reported later than the category's reporting period.	24
grace_boundary	Manual Review	Warranty ended but the claim is inside the grace period.	83
possible_duplicate	Manual Review	The invoice was already used on another claim.	31
missing_documents	Manual Review	Receipt missing, or two or more supporting documents missing.	149
serial_unverifiable	Manual Review	Serial mismatch that cannot be verified (evidence incomplete).	1
ambiguous_cause	Manual Review	Excluded cause without a confirming diagnosis, or unknown cause with low diagnostic confidence.	130
repeat_repairs	Manual Review	Previous repairs at or above the policy's repeat-repair threshold.	36
covered_defect	Valid Claim	Covered fault, inside the warranty, complete evidence, consistent data.	510

Claim ID and image filename mapping (first rows)

claim_id	split	claim_class	variation	filename	relative_path
CLM-00055	train	Invalid Claim	1	CLM-00055_v1.jpg	train/invalid/CLM-00055_v1.jpg
CLM-00055	train	Invalid Claim	2	CLM-00055_v2.jpg	train/invalid/CLM-00055_v2.jpg
CLM-00910	train	Manual Review	1	CLM-00910_v1.jpg	train/manual_review/CLM-00910_v1.jpg
CLM-00910	train	Manual Review	2	CLM-00910_v2.jpg	train/manual_review/CLM-00910_v2.jpg
CLM-01375	train	Invalid Claim	1	CLM-01375_v1.jpg	train/invalid/CLM-01375_v1.jpg
CLM-01375	train	Invalid Claim	2	CLM-01375_v2.jpg	train/invalid/CLM-01375_v2.jpg
CLM-00083	train	Valid Claim	1	CLM-00083_v1.jpg	train/valid/CLM-00083_v1.jpg
CLM-00083	train	Valid Claim	2	CLM-00083_v2.jpg	train/valid/CLM-00083_v2.jpg

Sample Claim Summary Cards

One training card per class. A card shows claim facts only — never the Python prediction, a confidence score or the final decision.

Claim summary
CLM-00004 · Consumer Electronics · Motherboard failure

✓ Within coverage 18 days left	✓ Reported in time 28 days · limit 30	✓ Manufacturing Defect covered cause · diag 0.75
✓ Receipt on file	✓ Evidence 3 of 3 supporting	✓ Serial matches registered = evidence
✓ Dates consistent purchase < fault < claim	✓ Invoice unique not seen before	✓ Repairs 0 previous

Purchased 15/02/2024 · claim 28/01/2026
Claim facts vs policy limits. Contains no model output or decision.

Valid Claim: CLM-00004_v1.jpg

Claim summary
CLM-00006 · Home Appliances · Thermostat failure

✗ Coverage ended 243 days ago (grace 14)	✓ Reported in time 4 days · limit 45	! Accidental Drop excluded cause · diag 0.19
✓ Receipt on file	✓ Evidence 3 of 3 supporting	✓ Serial matches registered = evidence
✓ Dates consistent purchase < fault < claim	✓ Invoice unique not seen before	✓ Repairs 1 previous

Purchased 26/06/2023 · claim 24/02/2026
Claim facts vs policy limits. Contains no model output or decision.

Invalid Claim: CLM-00006_v1.jpg

Claim summary
CLM-00001 · Consumer Electronics · Wi-Fi/Bluetooth failure

✓ Within coverage 826 days left	✓ Reported in time 5 days · limit 30	✓ Manufacturing Defect covered cause · diag 0.79
✗ Receipt missing	✓ Evidence 3 of 3 supporting	✓ Serial matches registered = evidence
✓ Dates consistent purchase < fault < claim	✓ Invoice unique not seen before	✓ Repairs 1 previous

Purchased 24/10/2025 · claim 21/07/2026
Claim facts vs policy limits. Contains no model output or decision.

Manual Review: CLM-00001_v1.jpg

3. Warranty policy files

Warranty rules are configuration, not code: one JSON file per category in [policies/](#), loaded by [src/rules/policy_store.py](#) and editable (with preview and version history) in Admin > Policies.

Consumer Electronics — [policies/consumer_electronics.json](#)

Field	Value
category	Consumer Electronics
policy name	Standard Consumer Electronics Warranty Policy
coverage duration months	12
standard terms months	12, 24
extended extra months	12
start conditions	Coverage starts on the purchase date shown on the verified invoice.
grace period days	7
claim reporting period days	30
covered faults	Screen flickering, Motherboard failure, Battery not charging, Speaker malfunction, Touch panel unresponsive, Wi-Fi/Bluetooth failure
excluded damage types	Accidental Drop, Water Ingress, Electrical Surge
exclusions	Liquid ingress or immersion, Screen shattering from drops, Power surge / overvoltage damage, Cosmetic casing wear, Commercial rental use
exclusion min diagnostic confidence	0.55
repeat repair review threshold	2
authorized service center required	True
repair conditions	Repairs must be carried out by an authorised service center using genuine parts.
replacement conditions	Unit replacement is considered only after a repeated failure of the same component.
mandatory documents	receipt
supporting documents	warranty_card, serial_photo, damage_photo
hard fail rules	WARRANTY_EXPIRED, UNAUTHORIZED_REPAIR, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING
manual review rules	GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED, SERIAL_UNVERIFIED, INCOMPLETE_EVIDENCE, UNKNOWN_CAUSE, REPEAT_REPAIRS, PRIOR_REPLACEMENT, UNCOVERED_FAULT
warning rules	NEAR_REPORTING_DEADLINE, WARRANTY_ENDING_SOON, SUPPORTING_DOCUMENT_MISSING

Home Appliances — [policies/home_appliances.json](#)

Field	Value
category	Home Appliances

Field	Value
policy name	Major Home Appliances Comprehensive Protection Policy
coverage duration months	24
standard terms months	24, 36
extended extra months	12
start conditions	Coverage starts on the purchase date shown on the verified invoice.
grace period days	14
claim reporting period days	45
covered faults	Compressor failure, Motor burnout, Thermostat failure, Drum spin malfunction, PCB failure, Water pump leakage
excluded damage types	Water Ingress, Accidental Drop
exclusions	External water / flood damage, Impact damage during relocation, Improper electrical supply rating, Pest infestation, Commercial heavy utilisation
exclusion min diagnostic confidence	0.55
repeat repair review threshold	2
authorized service center required	True
repair conditions	Repairs must be carried out by an authorised service center using genuine parts.
replacement conditions	Unit replacement is considered only after a repeated failure of the same component.
mandatory documents	receipt
supporting documents	warranty_card, serial_photo, damage_photo
hard fail rules	WARRANTY_EXPIRED, UNAUTHORIZED_REPAIR, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING
manual review rules	GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED, SERIAL_UNVERIFIED, INCOMPLETE_EVIDENCE, UNKNOWN_CAUSE, REPEAT_REPAIRS, PRIOR_REPLACEMENT, UNCOVERED_FAULT
warning rules	NEAR_REPORTING_DEADLINE, WARRANTY_ENDING_SOON, SUPPORTING_DOCUMENT_MISSING

Industrial Tools — [policies/industrial_tools.json](#)

Field	Value
category	Industrial Tools
policy name	Heavy Duty Industrial Equipment & Power Tools Policy
coverage duration months	24
standard terms months	24, 36
extended extra months	12
start conditions	Coverage starts on the purchase date shown on the verified invoice.

Field	Value
grace period days	14
claim reporting period days	30
covered faults	Armature burnout, Seal failure, Gearbox seizure, Chuck bearing failure, Trigger switch failure
excluded damage types	Accidental Drop, Water Ingress
exclusions	Tool dropped from height, Submersion in water, slurry or solvent, Overload beyond rated torque, Consumable wear (brushes, chuck teeth, cords)
exclusion min diagnostic confidence	0.55
repeat repair review threshold	2
authorized service center required	True
repair conditions	Repairs must be carried out by an authorised service center using genuine parts.
replacement conditions	Unit replacement is considered only after a repeated failure of the same component.
mandatory documents	receipt
supporting documents	warranty_card, serial_photo, damage_photo
hard fail rules	WARRANTY_EXPIRED, UNAUTHORIZED_REPAIR, EXCLUDED_DAMAGE, SERIAL_MISMATCH, LATE_REPORTING
manual review rules	GRACE_PERIOD, EXCLUDED_DAMAGE_UNCONFIRMED, SERIAL_UNVERIFIED, INCOMPLETE_EVIDENCE, UNKNOWN_CAUSE, REPEAT_REPAIRS, PRIOR_REPLACEMENT, UNCOVERED_FAULT
warning rules	NEAR_REPORTING_DEADLINE, WARRANTY_ENDING_SOON, SUPPORTING_DOCUMENT_MISSING

4. Security and access-control design

Model

Every request passes three checks, in order. Any failure denies.

1. **Permission** — the role holds the permission (`config/rbac.json`). Nothing is implied: deny by default.
2. **Scope** — the record is inside the role's reach: `own`, `service_center`, `assigned_or_queue`, `all`. Out-of-scope records return **404**, so IDs can't be probed.
3. **Constraints** — separation of duties, claim state machine, edit locks, admin self-protection.

The same scope drives list queries (`scoped_claims`) and single-record checks (`authorize_object`), so a record hidden on the detail page can never leak through search, dashboards or CSV export.

Roles

Role	Reaches	Can't
Customer	own products, claims, documents	see others' records, edit a submitted claim, review
Service-center staff	records of their service center	other centers, decisions, policies
Claim reviewer	unassigned queue + claims assigned to them	decide a claim they filed, skip states, override without a reason
Administrator	everything, read	decide claims (config owners don't adjudicate), change own role, remove the last admin

Loopholes found in v1 and how v2 closes them

#	v1 behaviour	v2
1	<code>/register</code> accepted <code>role=Admin</code>	public sign-up creates customers only; others are invited
2	role read from the session cookie; role change / disable took effect only after logout	user reloaded from DB each request; <code>session_version</code> revokes old sessions
3	<code>/claims/documents/<id>/correct-data</code> had no ownership check (any user could edit any OCR data)	<code>authorize_object("document.correct_ocr", doc)</code>
4	object checks existed only for customers (<code>if role == customer and curr_user ...</code>)	scope check for every role
5	staff and reviewers saw every product, claim and document	service-center and assignment scopes
6	reviewer could approve own claim, re-open closed claims, override without reason	SoD + transition table + 20-char reason
7	login said whether an email exists; no logout; session not rotated	one generic message, 5-try logout, <code>session.clear()</code> on login
8	a new route was public unless someone added a decorator	before-request hook blocks any unguarded endpoint
9	no CSRF on state-changing forms	Flask-WTF CSRFProtect (<code>src/app_security.py</code>)

Evidence

[tests/test_rbac_policy.py](#) (32 cases, pure) and [tests/test_rbac_routes.py](#) (app-level: unguarded-route inventory, escalation, enumeration, session revocation, role x endpoint matrix, IDOR).

5. Demonstration video

Video: [demo_video/AssureX_Demo.mp4](https://github.com/Saba1512006/assurex-claim-engine/blob/demo-video/AssureX_Demo.mp4) — one file, 13 min 37 s, 1920 × 1080, 30 fps, H.264 with AAC audio, recorded from the running application against a freshly seeded database. Download: https://github.com/Saba1512006/assurex-claim-engine/blob/demo-video/AssureX_Demo.mp4

- **Narration:** English, generated with the open-source Kokoro text-to-speech model (voice [af_heart](#)) from the script below, and placed at the timestamps shown. Declared in [AI_USAGE.md](#) §3.
- **On screen:** every narrated step also has a caption, and the element being discussed is outlined, so the video can be followed with the sound off.
- **All four roles** are shown signed in: customer (Parts 2–3), claim reviewer (Part 5), service-center staff (Part 6) and administrator (Parts 4, 7 and 8).
- **All 11 SRS demonstration cases** are shown in Part 4, each decided live: valid, invalid, manual review, expired warranty, missing document, duplicate, contradiction, serial mismatch, unauthorised repair, boundary date and model disagreement.
- **Nothing is staged:** every claim, prediction and decision is produced live by the application; the recording is unedited.

SRS §1.10 item 13 — where each required step is shown

Required in the video	Time
User registration or login	1:44
Product registration	2:12
Warranty registration	2:37
Claim creation	2:52
Receipt and document upload	3:06
OCR extraction	2:18
Extracted-data verification	2:29
Data preprocessing	3:54
Python prediction and confidence for all three classes	3:54
Claim Summary Card generation	4:42
Teachable Machine classification and confidences	4:05
Comparison of both predictions and confidence difference	4:15
Warranty-rule execution	4:25
Missing-document detection	6:25
Contradiction detection	6:54
Duplicate-claim detection	6:40
Manual-review routing	8:15
Reviewer comments or override	8:35
Administrator dashboard	10:19
Claim status tracking	4:58
Report generation	5:06
One valid claim	5:35
One invalid claim	5:45
One manual-review claim	5:58

Required in the video	Time
One tricky boundary case	7:34
One model-disagreement case	7:49

Chapters

Time	Chapter
0:04	Title card: Warranty claims, checked twice
0:29	PART 1 · The live check
1:32	PART 2 · Customer journey
3:36	PART 3 · The decision
5:14	PART 4 · The 11 SRS demonstration cases
8:00	PART 5 · Manual review (claim reviewer)
8:54	PART 6 · Service-center desk (service-center staff)
10:03	PART 7 · Administration (administrator)
11:29	PART 8 · Beyond the requirements
13:11	Closing card: Two models. One rulebook.

Script

Time	On screen	Narration
0:05	Title card: Warranty claims, checked twice	Hi, and welcome to Assure X, a warranty claim engine built for TechWiz 7. Every claim is checked twice: once by a Python machine learning model, and once by a Google Teachable Machine image model. A configurable warranty policy engine then turns those results into a decision, and whenever the system is unsure, a person decides.
0:30	PART 1 · The live check	Let's start with the landing page, and its live check.
0:37	The live check runs a real sample claim through the whole pipeline	Right on the home page, this live check runs a real sample claim through the complete pipeline, the same code the product uses.
0:45	Clean fault: both models say Valid, no rule blocks it, so the claim is Likely Valid	Here's a clean fault. Both models predict Valid Claim, and no warranty rule blocks it, so the recommendation is Likely Valid.
0:57	Liquid damage: an excluded cause confirmed by a technician gives Likely Invalid	Now liquid damage. That cause is excluded by the policy, and a technician confirmed it, so a hard-fail rule makes it Likely Invalid.
1:08	Filed four days into the grace period: Uncertain Result, so a person reviews it	And this one was filed four days into the grace period. Confidence drops, the result is uncertain, so the claim goes to a person instead of being auto-decided.
1:22	Behind the card: two models feed the decision core, the policy engine and the outcome	Behind the card, you can see the flow: the Python model and the vision model feed a decision core, which applies the warranty policy and produces the outcome.
1:33	PART 2 · Customer journey	Part two: the customer journey. We'll register, add a product with O C R, and file a claim from start to finish.
1:44	User registration: every field is validated in the browser and on the server	First, a new customer creates an account. Every field is validated, both in the browser and again on the server.
1:56	Sign in with the new account	Now we sign in with the new account.

Time	On screen	Narration
2:02	The customer dashboard: products, warranties, claims and pending actions	This is the customer dashboard. Products, warranties, claims, and anything waiting on the customer, all in one place.
2:12	Product registration: upload the purchase receipt and OCR reads it	To register a product, the customer simply uploads the purchase receipt, and O C R reads it.
2:18	OCR fills in product, model, serial number, purchase date, price, retailer and invoice number	In a moment, the product name, model, serial number, purchase date, price, retailer and invoice number are all filled in automatically.
2:29	The customer checks each extracted field before saving	The customer checks each extracted field before anything is saved.
2:37	The category sets the warranty length: standard or extended cover	The category sets the warranty length, with standard or extended cover.
2:45	The product and its warranty: coverage dates, status and documents	And here's the registered product, with its warranty coverage dates, status and documents.
2:52	Claim creation: choose the product	Now let's file a claim. Step one: choose the product.
2:58	Fault details: category, cause, start date and a description	Step two: the fault. Its category, the cause, when it started, and a short description.
3:06	Upload the receipt: it is read again and compared with the registered product	Step three: documents. The receipt is read again, and compared with the product on record.
3:14	OCR check: the serial number on the receipt matches the registered product	The serial number on the receipt matches the registered product, so this check passes.
3:19	Supporting evidence: warranty card, damage photo and serial-number photo	We add the supporting evidence: the warranty card, a photo of the damage, and a photo of the serial number.
3:29	Review the claim, then submit	Finally, a quick review, and we submit.
3:37	PART 3 • The decision	Part three: the decision. This is the heart of the system.
3:44	The verdict: the recommendation, both models and the consistency check	Within a couple of seconds, the claim is decided. At the top is the recommendation, and the time the whole pipeline took.
3:54	Python model: confidence for Valid, Invalid and Manual Review	On the left is the Python model. The claim's data is pre-processed into features, and the model gives a confidence for each of the three classes.
4:05	Teachable Machine: classifies the Claim Summary Card, with its own three confidences	On the right is Google Teachable Machine. It classifies an image, the Claim Summary Card, and gives its own three confidences.
4:15	Comparison: do the classes match, the confidence difference, and the consistency status	In the middle, the two models are compared: do the predicted classes match, how far apart are their confidences, and are they consistent?
4:25	Why this decision: the warranty rules and the decision-table row that matched	Below that, why this decision. Every warranty rule from the category policy is listed, with the decision table row that matched.

Time	On screen	Narration
4:35	What moved the Python model: each input's effect on the prediction	For explainability, this shows which inputs moved the Python model, and by how much.
4:42	What the image model saw: claim facts only, never a prediction or a decision	And this is exactly what the image model saw. The card shows claim facts only, never a prediction or a decision. The shaded tiles show which parts drove its answer.
4:58	Claim status tracking: eight stages from filing to closing	The customer can track the claim through eight stages, from filing to closing.
5:06	Report generation: a PDF with the claim, evidence, both predictions, rules and decision	And with one click, a P D F report is generated, with the claim, the evidence, both predictions, the rules and the decision.
5:15	PART 4 • The 11 SRS demonstration cases	Part four: all eleven demonstration cases from the requirements, each one decided live by the application.
5:35	Valid claim: both models agree Valid and no rule blocks it: Likely Valid	One. A valid claim. Both models agree it's valid, no rule blocks it, so it's Likely Valid.
5:45	Invalid claim: an excluded cause confirmed by a technician: Likely Invalid	Two. An invalid claim. Water damage is excluded by the policy, and a technician confirmed it, so a hard-fail rule makes it Likely Invalid.
5:58	Manual-review claim: the cause is unknown, so a person has to look at it	Three. A manual review claim. The cause is unknown and the diagnosis is inconclusive, so a review rule sends it to a person.
6:11	Expired warranty: it ended 195 days before the claim, far past the grace period	Four. An expired warranty. The cover ended a hundred and ninety-five days before the claim, far past the seven-day grace period, so it's Likely Invalid.
6:25	Missing documents: the mandatory receipt is missing, so the claim waits	Five. Missing document detection. The mandatory receipt isn't there, so the claim waits for more information, and the customer is told exactly what's needed.
6:40	Duplicate detection: the same documents were already used in another claim	Six. Duplicate detection. The same warranty card and invoice were already used in another claim, so this one is flagged as a possible duplicate and sent for review.
6:54	Contradiction detection: the fault is dated before the purchase date	Seven. Contradiction detection. The fault is dated before the product was even bought, which is impossible, so the claim is routed to review.
7:08	Serial-number mismatch: the receipt shows a different serial than the registered unit	Eight. A serial number mismatch. The serial on the receipt doesn't match the registered unit, so the claim is Likely Invalid.
7:19	Unauthorised repair: the unit was repaired at an unauthorised service center	Nine. An unauthorised repair. The repair history shows work by an unauthorised service center, so the policy's hard-fail rule makes it Likely Invalid.
7:34	Boundary case: filed four days into a seven-day grace period	Ten. A tricky boundary case. It was filed four days into a seven-day grace period. The confidence is low, so the result is uncertain, and a person reviews it.
7:49	Model disagreement: Python says Invalid, Teachable Machine says Valid, so a person decides	And eleven. A model disagreement. Python says invalid, Teachable Machine says valid. When the models disagree, the system never guesses. A person decides.
8:01	PART 5 • Manual review (claim reviewer)	Part five: manual review.
8:15	The reviewer queue: filters, risk levels and waiting time	Claims that need a person land in the reviewer's queue, with filters and risk levels.

Time	On screen	Narration
8:23	The workbench: evidence, both models' scores and the warranty facts side by side	Let's open the disagreement case. The workbench puts the evidence, both models' scores and the warranty facts side by side.
8:35	Approve with a written comment, and a written reason for the override	The reviewer approves it, and writes a comment. Because this overrides the automated result, a written reason is required.
8:46	The decision is recorded, the customer is notified and the status changes	The decision is recorded, the customer is notified, and the claim's status changes. The override is kept in the audit trail.
8:55	PART 6 · Service-center desk (service-center staff)	Part six: the service center desk, the fourth role.
9:08	Service-center staff: only this center's claims and products, with the customer on every row	Service center staff sign in to their own desk. They see only the claims and products of their service center, with the customer's name on every row.
9:21	Staff file claims for walk-in customers, with the technician's diagnosis recorded	Staff can file a claim on behalf of a walk-in customer. This one was filed for Usman Tariq by Sana Malik, and the technician's diagnosis and confidence are recorded with it.
9:38	In the claim form, each product shows its owner, and the diagnostic confidence is required	In the claim form, every product shows its owner. And for staff, the technician's diagnostic confidence is a required field.
9:52	Staff record repairs; an unauthorised repair is flagged and checked by the warranty rules	Staff also record repairs on the product. This unit was repaired at an unauthorised center, so it's flagged here, and the warranty rules take it into account on every future claim.
10:04	PART 7 · Administration (administrator)	Part seven: administration.
10:19	Administrator dashboard: claims, automation rate, model agreement and workload	The administrator dashboard shows claim volume, the automation rate, model agreement and the review workload.
10:29	Charts: claims per week, recommendations, model agreement and confidence	The charts break down claims per week, recommendations, model agreement and confidence.
10:38	Analytics: frequent faults, rejection reasons, category trends and model evidence	Analytics shows the most frequent faults, the reasons claims are rejected, trends per category, and the evidence for both models.
10:49	Warranty policies are configuration: coverage, grace periods, exclusions and rule severity	Warranty policies are configuration, not code. Coverage, grace periods, excluded causes and rule severity can all be changed here, with version history.
11:02	Models: both installed with versions; the Teachable Machine export is uploaded and evaluated here	On the models page, both models are installed with their versions. A new Teachable Machine export can be uploaded and evaluated right here.
11:14	The audit trail: every login, upload, prediction, decision and override	The audit trail records every login, upload, prediction, decision and override.
11:22	Access control: roles, record scopes, invitations and blocked attempts	And access control manages roles, which records each role can see, invitations, and blocked attempts.

Time	On screen	Narration
11:30	PART 8 · Beyond the requirements	Part eight: going beyond the requirements.
11:37	What-if simulator: try a stricter threshold on 225 stored test decisions	The what-if simulator lets an administrator try a stricter confidence threshold on all 225 stored test decisions, before changing anything.
11:49	Automation rate, review load and accuracy update instantly; nothing is saved until Apply	Watch the automation rate, the review load and the accuracy update instantly. Nothing is saved until you press apply.
11:59	Batch evaluation: run uploaded claim records through the real pipeline	Batch evaluation runs a file of claim records through the real pipeline, chunk by chunk.
12:06	30 unseen test claims evaluated: accuracy against the labels, with a downloadable CSV	Thirty unseen test claims, evaluated, with accuracy against their labels, and a downloadable C S V.
12:16	Policy changes are previewed and versioned before they apply	Policy changes are previewed and versioned. And if a change affects a limit shown on the card, the system warns that the image model needs retraining.
12:30	Input safety: names take no digits and phone numbers take no letters	Input safety: names can't contain digits, and phone numbers can't contain letters. This is enforced in the browser and on the server.
12:41	Sign-in protection: five wrong passwords pause the account for one minute	Sign-in is protected too. After five wrong passwords, the account is paused for one minute, with a live countdown.
12:52	The public model card: metrics, confusion matrices and errors for both models	Finally, the public model card. Both models' metrics, confusion matrices and errors, read directly from the evaluation files.
13:03	The technical blog, also published on Medium	And there's a technical blog explaining how it was built, also published on Medium.
13:12	Closing card: Two models. One rulebook.	The Python model reaches eighty-nine point three percent, and Teachable Machine ninety-three point eight percent, on two hundred and twenty-five unseen test claims. Backed by two hundred and ninety-five automated tests. Two models, one rulebook, and a person whenever it matters. Thank you for watching.

6. AI tool usage declaration

Required by SRS §1.8 (item 9) and deliverable 15. Every AI tool used during development is listed below. **The final claim decision is never produced by a generative-AI service:** it comes from the team's Python classification model, the Google Teachable Machine model, the warranty rule engine and the decision table running inside this application. No external AI API is called at runtime.

Correction. An earlier version of this file stated that no AI code generators were used. That is not accurate for the v2 upgrade described below, so this declaration replaces it.

1. Google Teachable Machine (required by the SRS)

Field	Declaration
Tool	Google Teachable Machine (teachablemachine.withgoogle.com), Google LLC
Purpose	Image classification of Claim Summary Cards into Valid Claim / Invalid Claim / Manual Review
Type of assistance	Browser-based transfer-learning of an image model on the team's training cards; TensorFlow Lite export
Files / modules affected	<code>model/teachable_machine/</code> (export + <code>labels.txt</code> + <code>evaluation.json</code>), consumed by <code>src/core/gtm_classifier_v2.py</code>
Status	Trained by the team in the browser on the v3 cards and installed as <code>gtm-514702678b8c</code> (test accuracy 93.8%); see <code>documentation/GTM_EVIDENCE.md</code>
Testing	<code>notebooks/evaluate_gtm.py</code> / Admin > Models > Run evaluation on the 225 validation and 225 test cards

2. Claude Code (Anthropic) — v2 upgrade

Field	Declaration
Tool	Claude Code (Anthropic), an AI coding assistant
Date	26 September 2026
Purpose	Integrate the team's v2 upgrade kit (leakage-free dataset, calibrated model, square cards, RBAC) into the application, fix the defects the kit identified, redesign the web interface, write tests and update documentation
Prompt / type of assistance	"Complete the project using the attached v2 upgrade kit, model card and SRS; implement everything fully with a top-quality design." Code generation, refactoring, test writing, documentation drafting, running the training scripts and the test suite
Files / modules affected	Almost every module: <code>src/</code> (core pipeline, rules, security, services, api), <code>templates/</code> , <code>static/</code> , <code>config/</code> , <code>policies/</code> , <code>dataset_generator/</code> , <code>notebooks/</code> , <code>database/seed.py</code> , <code>tests/</code> , <code>reports/</code> , <code>documentation/</code> , <code>README.md</code> . Generated data: <code>data/raw</code> , <code>data/splits</code> , <code>data/summary_cards</code> , <code>model/python_model/</code>
What it did not do	Train the Teachable Machine model (requires the browser tool); decide any claim at runtime; invent metrics (every number in the docs is read from files the scripts produce)
Testing performed by the tool	192 automated tests (<code>python -m pytest -q</code>), a page-by-page render check of every screen for all four roles, browser screenshots, and an end-to-end seed that submits the 11 demonstration claims through the live pipeline

2b. Claude Code (Anthropic) — frontend rebuild

Field	Declaration
Tool	Claude Code (Anthropic), an AI coding assistant
Date	27 September 2026

Field	Declaration
Purpose	Rebuild the web interface from the team's written brief, add the explainability, simulation and batch features it asked for, and test them
Prompt / type of assistance	The team's <i>AssureX Frontend Master Prompt</i> (PDF, 10 parts, prepared by Muhammad Azhar Nawaz), plus the instruction to implement all of it together with three model-explanation additions. Code generation, refactoring, test writing, running the test suites, browser screenshots, accessibility and Lighthouse audits
Files / modules affected	templates/ (every page), static/css/ , static/js/ , src/api/api.py , src/api/errors.py , src/services/ (verdict , whatif , batch , model_card_service , receipt_scan , paging), src/core/explain_models.py , src/core/pipeline.py (stored verdict payload and stage timings), src/app_security.py , tests/ (contract , route-matrix and tests/e2e/ browser tests), scripts/subset_icons.py , README.md , documentation/PROJECT_REPORT.md
What it did not do	Change the Claim Summary Card design or either model (tests re-render the test cards and fail if Teachable Machine accuracy could drop; the batch evaluator reproduces 89.3% and 93.8% on the test split); decide any claim at runtime
Testing performed by the tool	225 automated tests including a route x role matrix over every guarded route; 13 browser journeys on a live seeded server with axe-core accessibility checks on 21 pages; Lighthouse on the public pages; screenshots at 1440, 1024 and 390 px

2c. Claude Code (Anthropic) — styling pass, input rules and deployment fixes

Field	Declaration
Tool	Claude Code (Anthropic), an AI coding assistant
Date	27–28 September 2026
Purpose	Visual polish requested by the team, form input rules, sign-in lockout countdown, landing-page hero background, dependency fixes for deployment, and updating the documentation to the final state
Prompt / type of assistance	The team's instructions, given one change at a time (for example "no numbers in name fields", "show a timer when sign-in is locked", "premium navbar and footer", "preloader like the logo"). Code generation, CSS, tests, browser screenshots, running the test suites
Files / modules affected	templates/ (navbar, footer, base, landing, auth, dashboards, claim detail, workbench, notifications), static/css/ , static/js/ (app.js , instrument.js , hero_engine.js , hero_engine_math.js), static/img/brand/ , src/rules/validator.py , src/api/auth.py , src/api/access.py , src/api/products.py , src/app.py , config/rbac.json (lockout minutes), requirements.txt , tests/ (test_input_rules.py , test_hero_engine_math.py , tests/js/ , tests/e2e/test_hero_engine.py), README and documentation/
What it did not do	Change either model, the dataset or the Claim Summary Card (the accuracy guards re-score both models on every test run: 89.3% and 93.8%); decide any claim at runtime
Testing performed by the tool	295 automated tests and 34 browser tests (overlap, contrast and pixel checks of the hero background at five widths, input rules, lockout countdown, accessibility with axe)

Required team entries (SRS: modifications, testing and verifier name)

The SRS requires the **team** to review, modify, test and understand AI output, and to name the member who verified it:

Module	Modifications made by the team	Tests run by the team	Verified by (team member)
Dataset generator & cards (dataset_generator/)	Saba Noor reviewed the generated dataset (class-independent attributes, 4% label noise, stratified 1,050 / 225 / 225 split) and the v3 card design adopted after the first Teachable Machine run	Checked that no validation or test claim appears in training (CSV or image) and that each card matches its CSV record, Claim ID and label	Saba Noor
Python model training (notebooks/train_python_v2.py)	Saba Noor ran the training script, reviewed the three-algorithm comparison and the selection on the validation split	Checked the test metrics (89.3% accuracy, 89.2% macro F1), the leakage audit and the saved-model accuracy guard in the test suite	Saba Noor
Teachable Machine training & evaluation	Saba Noor trained the image model in the browser on the 2,100 training cards (epochs 50, batch 16, learning rate 0.001), retrained after the card redesign (v2 → v3), and installed the export through Admin > Models	Evaluated on the 225 validation and 225 test cards (93.8% test accuracy, 97.3% validation) and reviewed the misclassified samples	Saba Noor
Evaluation pipeline (src/core/)	Muhammad Sami wrote the first pipeline (OCR, comparison, decision engine) before the v2 upgrade; the team kept the v2 decision table and thresholds after review	Deployed and ran the app on PythonAnywhere and locally; Saba Noor compared the live-check results with the model card figures	Muhammad Sami, Saba Noor
Rule engine & policies (src/rules/ , policies/)	Muhammad Sami wrote the original rule set, multi-category policies, serial verification and contradiction/duplicate checks; kept as the basis of the config-driven v2 rules	Muhammad Sami created the sample_claims/ suite covering the SRS demonstration cases; Muhammad Sami ur Rehman ran all 11 through the app and reported the defects found	Muhammad Sami, Muhammad Sami ur Rehman
Security & access control (src/security/ , config/rbac.json)	Saba Noor reviewed the role permissions and asked for the one-minute lockout and the server-side input rules	Signed in as each of the four roles and checked what each can open; tested the lockout countdown and the rate-limit page	Saba Noor
Web interface (templates/ , static/)	Muhammad Sami tuned the live check card height and hero padding (28 Sept); Saba Noor requested and reviewed every page change	Saba Noor ran the app locally on Windows and checked each page for all four roles; Muhammad Sami ur Rehman tested the pages in Chrome and on mobile; Muhammad Ghanyan checked the mobile view	Saba Noor, Muhammad Sami ur Rehman
Frontend rebuild features (verdict, workbench, what-if, batch, model card)	Built from the team's written brief; Muhammad Ghanyan's Figma designs for the login and dashboard pages and colour palette guided the look	Muhammad Sami ur Rehman ran the 11 sample claims through the app and tested the pages in Chrome and on mobile; Muhammad Ghanyan checked the mobile view	Saba Noor, Muhammad Sami ur Rehman
Styling pass, input rules, lockout countdown, hero background (27–28 Sept)	Saba Noor specified each change and asked for corrections after testing (menu scrolling, notifications link, footer layout, hero placement, text clipping, lockout time)	Typed digits into name fields and letters into phone fields, locked an account with five wrong passwords and watched the countdown, checked the landing page at several window sizes	Saba Noor
Documentation & report	Saba Noor corrected links, figures and wording; the blog was published on Medium from her account	Checked every figure against reports/ and the model files	Saba Noor

3. Other tools

Tool	Use
scikit-learn, pandas, NumPy, Pillow, ReportLab, pdfplumber, Tesseract (pytesseract)	Libraries used by the application (not AI assistants)
Chart.js, Bootstrap Icons, Alpine.js (CSP build), IBM Plex fonts	Front-end libraries and fonts, self-hosted in static/vendor/
Playwright, axe-core (axe-playwright-python), Lighthouse	Browser testing, accessibility and performance audits (development only)
Mermaid	Rendering the report diagrams from documentation/diagrams/*.mmd
Kokoro text-to-speech (open-source, voice af_heart)	English narration of the demonstration video, generated from the script in documentation/DEMO_VOICEOVER.md ; the team wrote and checked the script against the application

Images: the demo "damage" and "serial" photos created by [database/seed.py](#) are synthetic drawings made with Pillow, labelled as such on the image. No AI-generated imagery is used in the application.

7. Development log

Entries record work completed, problems encountered, model failures, changes made and tests performed. They are written from the Git history ([git log](#)); team members can add their own notes under each day.

Commit activity by day

Day	Commits	Accounts
23 September 2026	3	Muhammad Sami
24 September 2026	18	Muhammad Sami
25 September 2026	15	Muhammad Sami
26 September 2026	19	Muhammad Sami (13), Saba1512006 (6)
27 September 2026	59	Saba1512006
28 September 2026	6	Muhammad Sami (3), Saba1512006 (3)

23 September 2026 — first working version

Work completed - Deployment files (unicorn, Procfile, [render.yaml](#)); architecture diagram; database state synced. - Product lookup mismatch fixed in the claim wizard; document upload inside the wizard. - LICENSE, the [sample_claims/](#) suite, screenshots and export fixes; password confirmation on sign-up.

24 September 2026 — SRS functional requirements

Work completed - Multi-role registration, profile management and navigation (SRS 1.6 i-ii); product registration and the common warranty interface (iii-iv); receipt/invoice upload and OCR extraction (v-vi); extracted-data verification and warranty tracking (vii-viii); expiry alerts and claim registration by staff and users (ix-x); claim information, fault evidence upload and repair history (xi-xiii). - Document organisation, data validation and pre-processing (xiv-xvi); algorithm comparison on the admin dashboard (xviii); Claim Summary Card endpoint, Teachable Machine preview and prediction comparison UI. - Confidence comparison with consistency thresholds; the warranty rule set; multi-category policies, serial verification from four sources and contradiction detection; missing-document and duplicate checks.

Problems encountered and fixed - Quick-login reference error; a Jinja2 syntax error in the claim inspection page; demo claims not kept for the reviewer workbench; test clean-up leaking between tests.

25 September 2026 — decisions, dashboards and deployment fixes

Work completed - Claim summary, preparation assistance and the final decision engine; decision explanation, manual-review workflow and reviewer override audit trail; claim tracking and notifications; admin duplicate alerts, search and analytics; reporting and audit coverage; model version tracking and error handling. - UI clean-up (navbar, signed-in welcome card, user-facing labels).

Problems encountered and fixed - PythonAnywhere startup failures: a missing [typing.Any](#) import and the [reportlab](#) import at start-up. - Cross-platform model loading; strict OCR receipt validation (unreadable invoice images refused); a 500 error in the intake wizard; OCR mock fallback.

26 September 2026 — v2 upgrade (AI-assisted, see AI_USAGE.md)

Work completed - Replaced the dataset generator: attributes sampled independently of the class, labels derived from the policy files, 4% label noise, IDs assigned after shuffling, stratified 1,050/225/225 split. - Retrained the Python model: three algorithms, selection on validation, sigmoid calibration, leakage audit, permutation importance, model card. Test accuracy 89.3% (macro-F1 89.2%, ROC-AUC 0.948). - Redesigned the Claim Summary Card (600×600, bundled fonts, visual encoding of every feature) and rendered 2,100 training / 225 validation / 225 test cards. - Replaced the scikit-learn stand-in for Teachable Machine with a TFLite runtime, an admin installer and an evaluation script. The browser training itself is still to be done. - Rule engine rewritten around config: 16 checks, severity per category in [policies/*.json](#); decision table and routing in [config/decision_policy.json](#); admin editors for both. - Integrated role- and record-scoped access control, login hardening, CSRF, rate limits, CSP, upload sniffing. New UI for every page.

Problems encountered - v1 scored 100%: [damage_type](#) was a relabelled class and claim IDs were ordered by class (leakage). - v1 selected the model on the test split; v1 probabilities were near 1.000, so every comparison was a "Strong Match". - Form options did not match training values; unknown values were silently encoded as zeros. - OCR relied on hard-coded retailer/product lists; [Invoice](#) was read as an invoice number. - Landscape cards were centre-cropped by Teachable Machine; fonts differed on Linux.

Model failures observed - Invalid claims predicted as Manual Review (8/75 on test); 5 of 14 missed Invalid claims are the rare late-reporting scenario. Covered by the LATE_REPORTING hard-fail rule. - Boundary claim (4 days into grace) gets 57% confidence → *Uncertain Result*; correctly routed to review.

Changes made after testing - Warnings made advisory (blocking rules belong in [manual_review_rules](#)); receipt made the only mandatory document, with rules for missing supporting documents — aligns the decision table with the policy. - Receipts with readable text but no invoice details are refused; scanned PDFs without text are accepted.

Tests performed - 192 automated tests passing; every page rendered for all four roles; browser screenshots desktop/mobile; seed run submitting the 11 demonstration claims through the live pipeline.

Teachable Machine, first run (card v2) - Trained in the browser on the 2,100 v2 cards (epochs 50, batch 16, learning rate 0.001) and installed as [gtm-ec95b4aa49b5](#). Test accuracy 54.2% (validation 52.9%, macro F1 45.1%): below the 85% target. - The TensorFlow 2.20 package on Windows no longer ships [tf.lite.Interpreter](#); the runtime uses [ai-edge-litert](#), which installed fine on Windows. - Root cause found with an offline replica of Teachable Machine's trainer ([notebooks/tm_replica_check.py](#)): frozen MobileNet features cannot separate the v2 card's thin grey shapes (replica 58–66%). - Card redesigned (v3, policy-relative colour tiles); replica test accuracy 92.9–94.2%. Training cards regenerated.

Teachable Machine, second run (card v3) - Retrained in the browser on the 2,100 v3 cards with the same settings; installed as [gtm-514702678b8c](#). - Test accuracy 93.8% (macro F1 93.8%), validation 97.3%, agreement with the Python model 92.9%. - Installing over the running model returned a 500 on Windows (memory-mapped .tflite could not be archived). Fixed by loading models from bytes; regression test added.

Accuracy guards - Tests now re-score both saved models on every run: Teachable Machine on test cards rendered with the current card code and policy files (must stay ≥ 85% and equal to the recorded 93.8%), the Python model on the test split (≥ 85%, equal to 89.3%). A deliberate one-colour change to the card dropped the image model to 77.8% and failed the test, as intended. - The policy editor warns when a limit drawn on the card is changed (the image model then needs retraining).

Requirement audit against SRS §1.10 - Added the standalone label encoder, preprocessing pipeline, feature schema, processed train/val/test CSVs, labels file, scenario definitions and all 225 sample test predictions (extracted from the saved model; a test proves they match it). - Excel (.xlsx) export next to CSV for claims, analytics, audit and search, with the same formula guard. - Test results file ([reports/test_results.txt](#)), team contribution record template, demo-video script, deployed-app testing guide in the README; a seeded demonstration claim where the two real models disagree. - Teachable Machine error analysis: 12 of its 14 test errors are deliberately flipped labels.

27 September 2026 — frontend rebuild, styling and input rules (AI-assisted, see [AI_USAGE.md](#))

Work completed - Teachable Machine v3 results recorded (93.8% test accuracy) with evaluation screenshots, misclassified samples and the comparison report; accuracy guards for both models in the test suite; technical blog published on Medium and linked; seeded model-disagreement case; demo video script. - Frontend rebuilt from the team's written brief: design tokens and app shell, verdict screen with agreement meter, landing live bench, model card, claim wizard with autosave and OCR split view, reviewer workbench, admin overview, what-if simulator, batch evaluator, policy editor with history, analytics, audit, access and models pages. - Styling pass: premium navbar, sign-in and register with the card scanner, dashboards with KPI cards and validated chart colours, claim detail and workbench, new AssureX logo, footer, back-to-top button, animated preloader, and a hero engine background that follows the live check. - Input rules on every form: no digits in names, cities, brands or retailers; no letters in phone or number fields; strict email format — enforced in the browser and on the server. - Sign-in pause shortened to one minute after five wrong passwords, with a live countdown on the sign-in and rate-limit pages.

Problems encountered and fixed - The Notifications item in the account menu opened the profile page; the account menu did not scroll on short screens; the admin menu had lost the Models, What-if and Batch links. - Training-card download failed on Windows. - A blurred, translucent card over the animated background made the live check slow enough to fail its browser test; the blur was removed. - The rate-limit (5 per minute) message conflicted with the "15-minute lockout" hint; both now say one minute.

Tests performed - 295 automated tests and 34 browser tests (route × role matrix, accessibility with axe, no overlap of the hero background with text at five screen widths, text contrast, input rules and lockout countdown).

28 September 2026 — deployment

Work completed - Live check card height and hero padding tuned on the landing page (Muhammad Sami). - Requirements: [tflite-runtime](#) on Linux and [ai-edge-litert](#) on Windows (Muhammad Sami). - The rate-limit page made compatible with older Flask-Limiter releases; scikit-learn pinned back to 1.9.1 so the saved Python model loads (Saba1512006). Work from a separate fork was merged into the main repository.

Problems encountered - On the PythonAnywhere free account both models showed as *Unavailable*: the account runs Python 3.10, whose newest scikit-learn (1.7.2) cannot load the model trained with 1.9.1, and the TFLite runtime was not installed. Creating a Python 3.11 virtualenv failed on the account's system image, and installing the TFLite runtime hit the 512 MB disk quota. - Render needed a payment card even for the free instance, so it was not used. - Claims on the live site therefore go to manual review (the designed fallback). The full pipeline runs locally with the pinned requirements; see *Known limitations* in the README.

Tests performed - 295 automated tests and 34 browser tests pass locally with Python 3.11 and scikit-learn 1.9.1 ([reports/test_results.txt](#)).

Open items

- Record and link the demonstration video (.mp4).
- Complete the team rows in [AI_USAGE.md](#) and [documentation/TEAM_CONTRIBUTIONS.md](#).
- Add the Teachable Machine project link to [documentation/GTM_EVIDENCE.md](#).